



RAGHU INSTITUTE OF TECHNOLOGY

(Autonomous)

(Approved by AICTE, New Delhi, Accredited by NBA & Re Accredited by NAAC with 'A+' grade & Permanently Affiliated to JNTU-GV Vizianagaram)

CODE REVIEW SYSTEM USING Generative AI

Under the esteemed guidance of
Dr. P. APPALA NAIDU
(Professor & HoD)

Presented by:
H. PRAVALIKA(213J1A0565)
G. SUNIL KUMAR(213J1A0551)
A. TARUNI(213J1A0501)
G. PRENITH SAI(213J1A0563)

ABSTRACT

An AI Assisted Code Review System is an innovative solution designed to automate and enhance the software code review process using generative AI. It employs specialized agents to handle critical aspects of code analysis, including syntax validation, security vulnerability detection, performance optimization, and adherence to coding best practices.

Built on OpenAI's GPT models, the system leverages Celery for distributed task orchestration, enabling asynchronous and efficient processing. FastAPI serves as the interface for seamless API integration, while Docker ensures scalability and ease of deployment.

By delivering detailed and actionable feedback, this system reduces the need for manual code reviews, accelerates development cycles, and improves overall code quality. It offers a robust, scalable, and developer-friendly approach to modernizing software development workflows.

Literature Survey

AI Code Review: Advancing code quality with AI-enhanced reviews

Link: <https://www.sciencedirect.com/science/article/pii/S2352711024000487>

Algorithms Used	Working	Drawback	Datasets Used
1. GPT-3.5-based NLP engine. 2. Neural networks for syntax and semantic analysis..	1. Code snippets are parsed for syntax/semantic discrepancies. 2. Suggestions are generated using a pre-trained language model. 3. Focused on integrating with IntelliJ IDEA for real-time analysis.	1. Reliance on GPT-3.5 limits domain-specific accuracy. 2. Limited support for programming languages other than those explicitly configured. 3. Potential overfitting due to fine-tuning on specific datasets.	1. Code repositories analyzed via IntelliJ IDEA plugin, leveraging GPT-3.5 for assessment.

Architecture Diagram

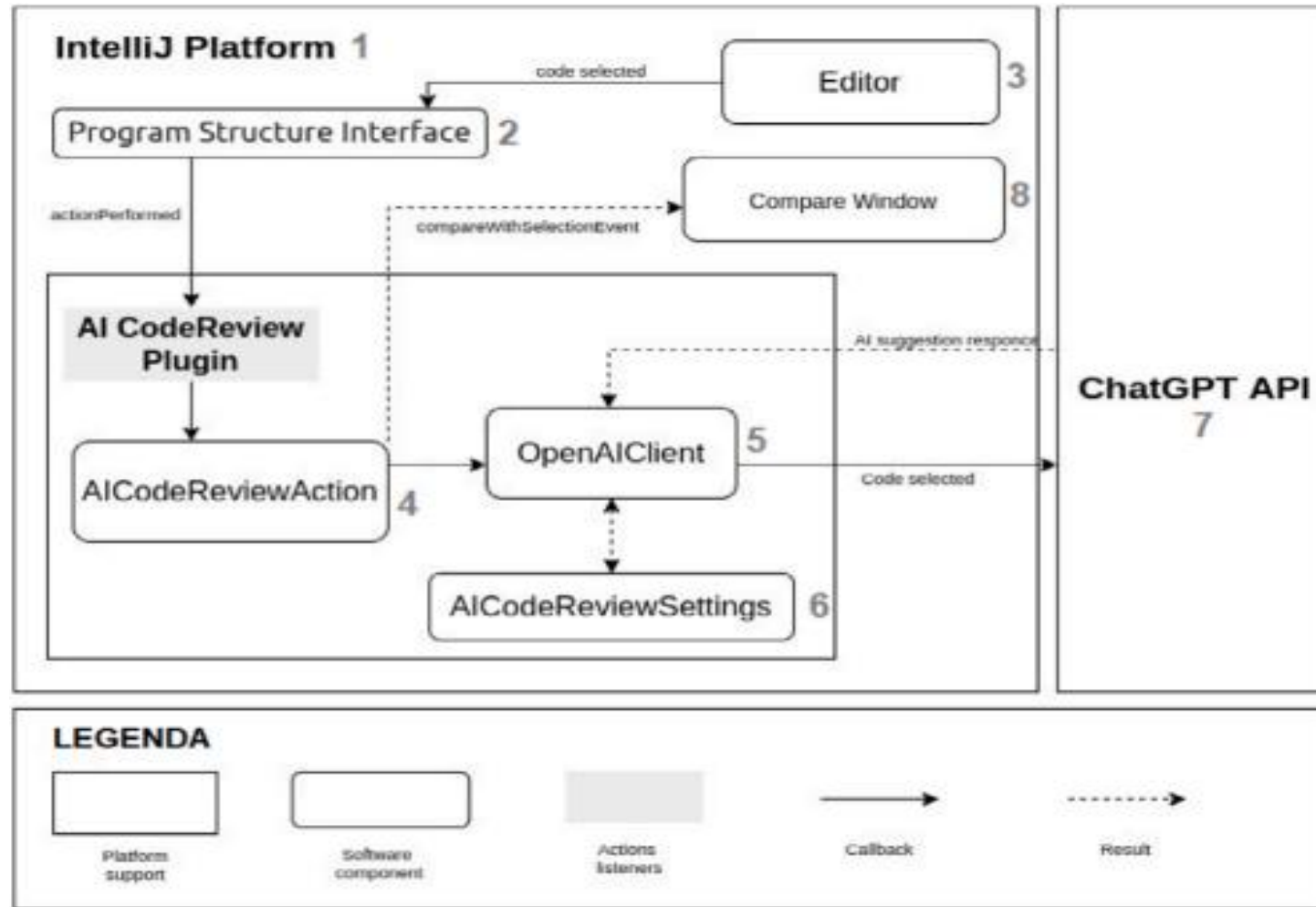


Fig-1: AI Code Review

Enhancing Software Quality through AI - Assisted Code Review

Link: <https://www.ijsr.net/archive/v12i2/SR24716230727.pdf>

Algorithms Used	Working	Drawback	Datasets Used
1. Rule-based systems combined with LLMs for best practice enforcement. 2. Hybrid approaches using style guide-specific datasets.	1. Verifies adherence to best practices and assists reviewers with contextual feedback. 2. Optimized for Google's large-scale industrial codebases.	1. Scalability issues in non-Google environments. 2. Requires extensive customization for non-standard best practices. 3. Partial automation; still heavily reliant on human reviewers for functional validation.	1. Historical code review data from AWS Cloud Infrastructure Development. 2. Includes open-source and enterprise-level datasets for validation.

Architecture Diagram

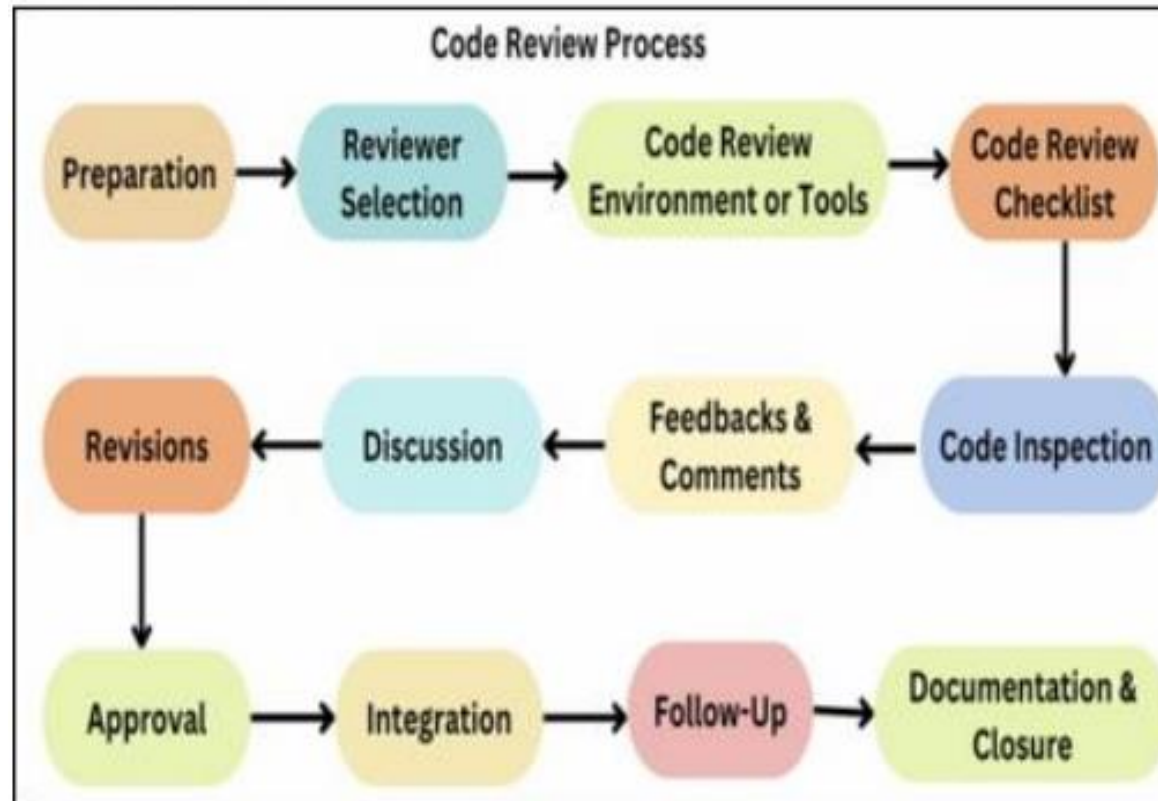


Fig-2: Enhancing Software Quality through AI - Assisted Code Review

A GPT-based Code Review System for Programming Language Learning

Link: <https://arxiv.org/abs/2407.04722v1>

Algorithms Used	Working	Drawback	Datasets Used
1. GPT-4 for personalized code reviews. 2. Dataset-enhanced prompts for accuracy in educational scenarios.	1. Collects learner-submitted code and highlights areas for improvement. 2. Reduces potential AI-assist cheating by focusing on submitted code rather than generating solutions.	1. Context limitations due to dataset dependency. 2. Slower in larger educational environments due to scalability issues. 3. Vulnerable to false negatives for non-standard code submissions.	1. Training data includes public repositories and educational code snippets tailored for teaching.

Architecture Diagram

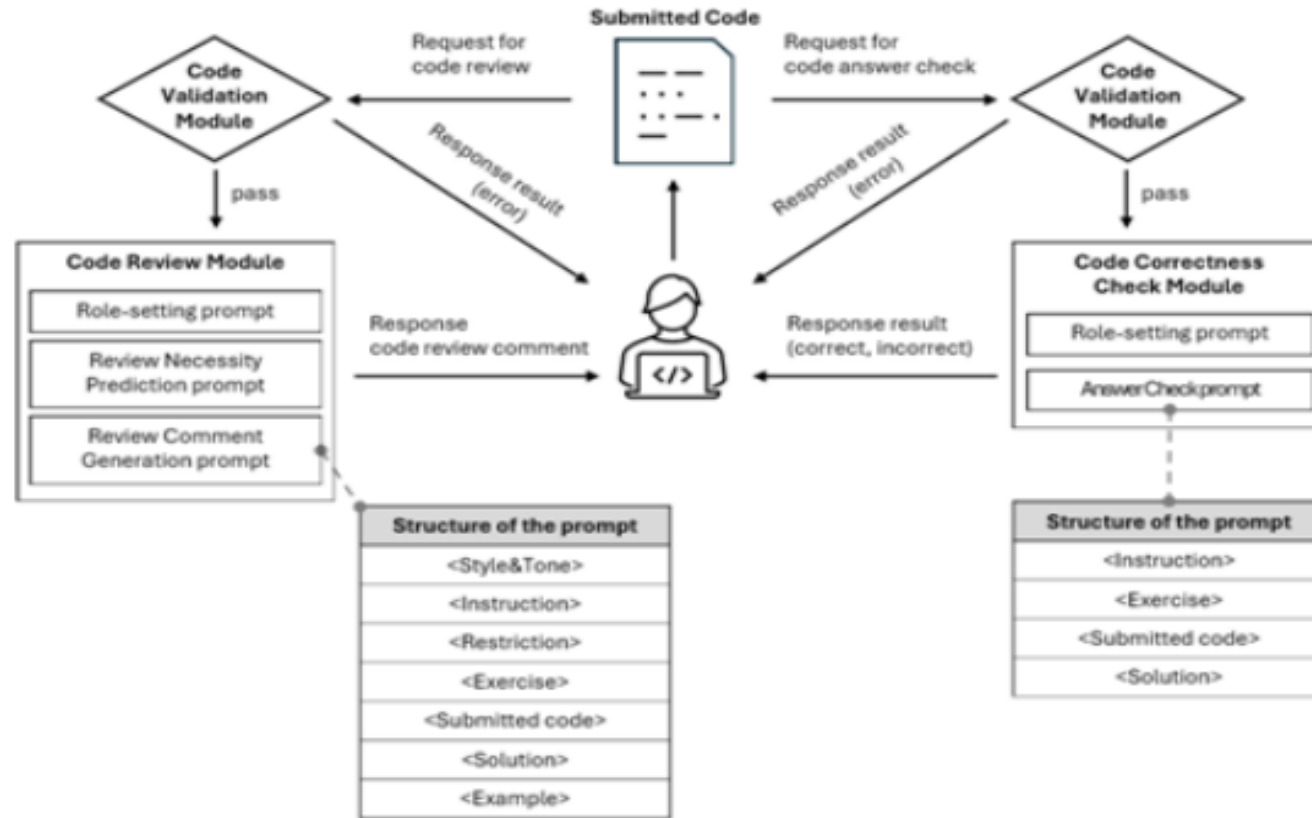


Fig-3: A GPT-based Code Review System for Programming Language Learning

AI-Assisted Assessment of Coding Practices in Modern Code Review

Link: <https://arxiv.org/abs/2405.13565>

Algorithms Used	Working	Drawback	Datasets Used
1. Large Language Models (LLMs) for natural language feedback. 2. Static analysis integration for auto-comment generation.	1. Generates automated comments based on best practice violations. 2. Enables reviewers to focus on logic rather than style violations.	1. High false positive rates for nuanced style issues. 2. Limited adaptability to evolving best practices in codebases. 3. Reviewer reliance on automated suggestions may reduce manual scrutiny.	1. Code submissions from diverse programming languages (C++, Java, Python, Go). 2. Likely includes open-source datasets and repositories.

Architecture Diagram

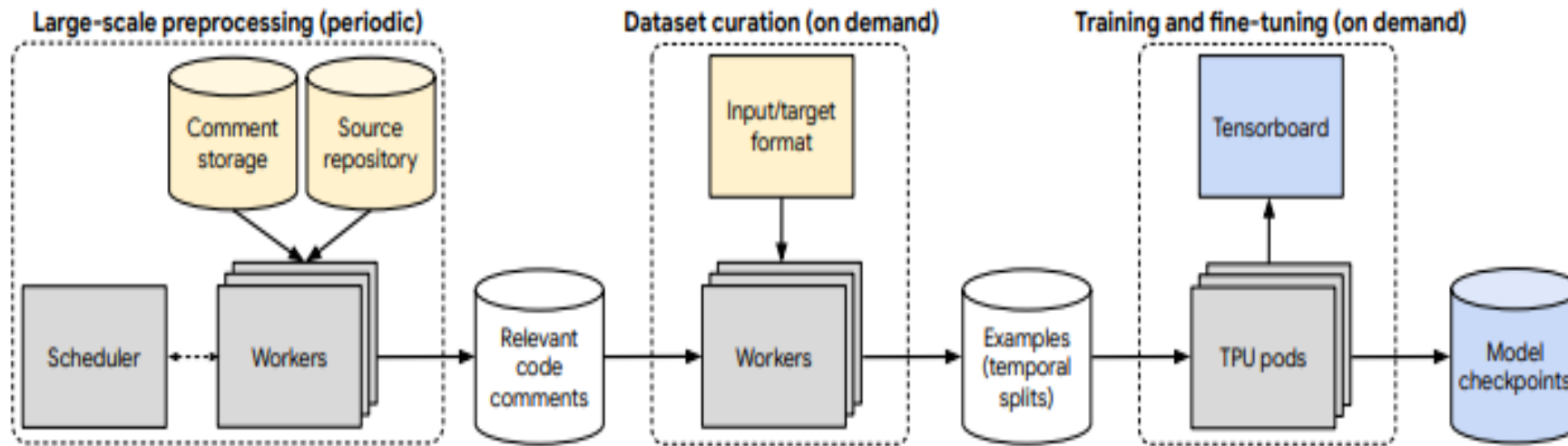


Fig-4: AI-Assisted Assessment of Coding Practices in Modern Code Review

AI-powered Code Review with LLMs

Link: <https://arxiv.org/abs/2404.18496>

Algorithms Used	Working	Drawback	Datasets Used
1. Transformer-based models for contextual analysis. 2. Multi-level embeddings for semantic understanding.	1. Detects code smells, predicts potential bugs, and suggests optimizations 2. Enhances developer education through detailed feedback mechanisms.	1. Over-reliance on pre-trained datasets can lead to biased recommendations. 2. Limited empirical validation against manual reviews in industrial settings. 3. Poor performance in detecting deeper architectural flaws.	1. Large code repositories from GitHub, bug reports, and documentation of best practices. 2. Dataset sourced via GitHub REST API for public repositories and historical pull requests.

Optimizing AI-Assisted Code Generation: Enhancing Security, Efficiency, and Accessibility in Software Development

Link: <https://arxiv.org/abs/2412.10953v1>

Algorithms Used	Working	Drawback	Datasets Used
1. Encoder-Decoder Transformer models (e.g., GPT-3, Codex, AlphaCode). 2. Static analysis tools integrated with LLMs.	1. AI generates, evaluates, and optimizes code snippets for security and efficiency. 2. Incorporates feedback loops to improve successive generations of code.	1. Generated code may contain logical errors despite syntactical correctness. 2. Struggles with domain-specific languages not included in training data. 3. High computation cost and dependency on GPU infrastructure.	1. Pre-trained on extensive repositories like GitHub and datasets used by LLMs (e.g., OpenAI Codex, AlphaCode).

Architecture Diagram

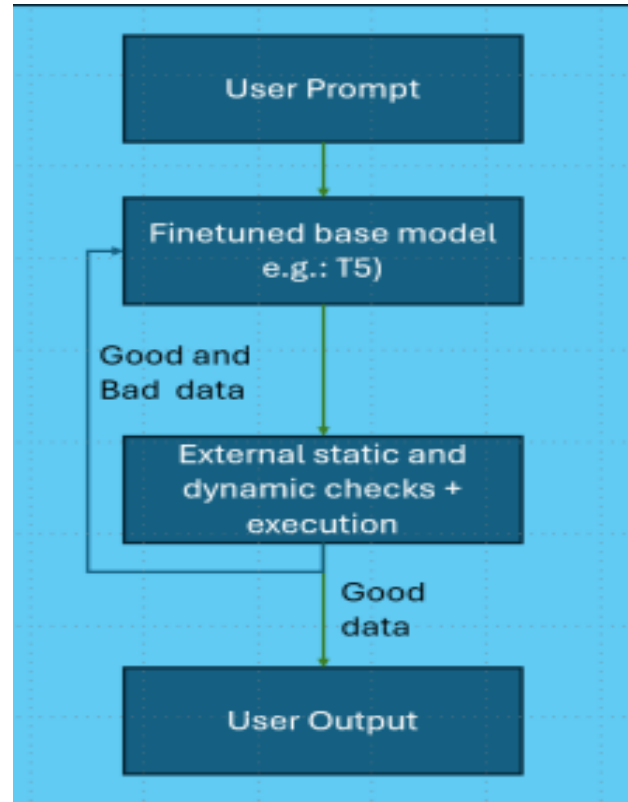


Fig-5: Optimizing AI-Assisted Code Generation: Enhancing Security, Efficiency, and Accessibility in Software Development

Existing System

Existing AI-based code review systems rely on single-purpose models or static analysis tools for identifying issues like syntax errors, security vulnerabilities, and performance bottlenecks. These systems often lack integration, leading to fragmented insights. They also struggle with real-time feedback, adaptability to evolving coding standards, and comprehensive error detection across multiple domains. Furthermore, some systems fail to address nuanced context-specific issues or provide actionable, user-friendly recommendations, limiting their utility for developers.

Proposed System

Our proposed AI Code Review System uses advanced Generative AI agents powered by OpenAI GPT models, each specializing in areas like syntax validation, security analysis, performance optimization, and coding best practices. By orchestrating these agents through Celery and integrating them via FastAPI, the system delivers consolidated, real-time feedback that is both actionable and easy to understand. Unlike traditional systems, it eliminates fragmented insights, adapts to evolving coding standards, and scales effortlessly with Dockerized deployment, providing developers with a unified and intelligent code review experience.

Datasets used:

Pre-trained GPT models: These are already equipped with knowledge from diverse codebases.

Real-time user inputs: Submitted during system use.

How our system overcome drawbacks of the existing system?

Drawback	What we are using	Explanation
Limited specialization in code reviews.	Multi-Agent Design with specialized agents.	Each agent (Syntax, Security, Performance, Best Practices) focuses on a specific task, ensuring thorough analysis.
Lack of actionable AI feedback.	Generative AI (GPT-4) with tailored prompts.	GPT-4 provides context-aware, detailed suggestions tailored to each aspect of the review.
Focus on single aspects of code quality.	Layered Review Approach.	Combines syntax checking, security analysis, performance optimization, and best practices enforcement for holistic feedback.
Inconsistent or redundant results.	Agent Orchestration Layer.	Combines and reconciles results from multiple agents into a unified, consistent response.
Complex deployment	Containerization with Docker	Provides a simple, portable setup for deployment across environments.

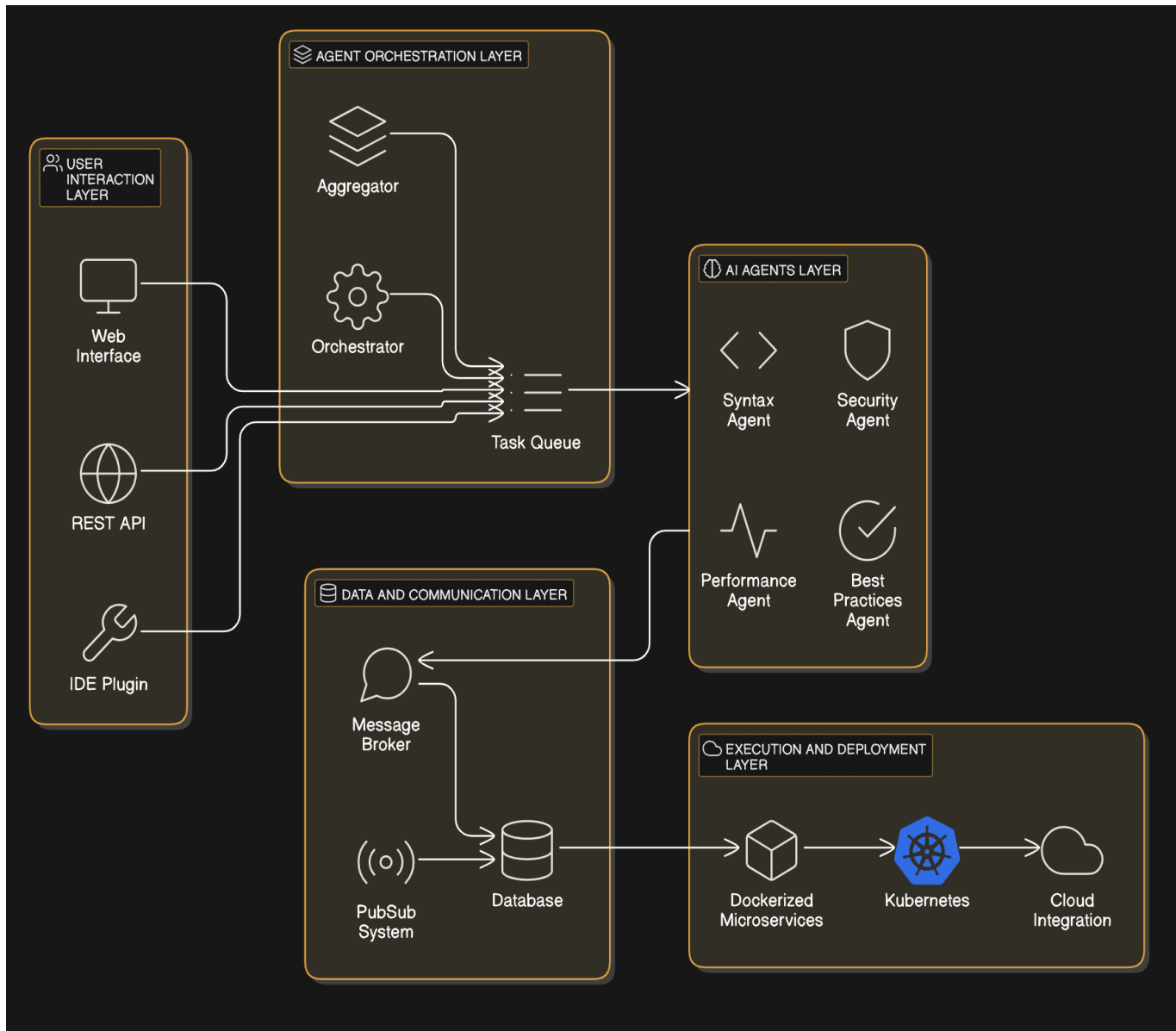


Fig-6: Architecture of code review system using GenAI

This diagram represents a modular software architecture with several interconnected layers:

1. User Interaction Layer: Interfaces like a web app, API, or IDE plugin let users interact with the system.

2. Agent Orchestration Layer: Manages tasks using an Orchestrator and Task Queue. It assigns work to specialized agents.

3. AI Agents Layer: Specialized agents (e.g., Syntax, Security, Performance, Best Practices) analyze and optimize tasks.

4. Data and Communication Layer: Handles data flow using a Message Broker, PubSub System, and a Database for storing and exchanging information.

5. Execution and Deployment Layer: Deploys the system using Dockerized Microservices, Kubernetes for scaling, and integrates with the cloud.

USE CASE DIAGRAM

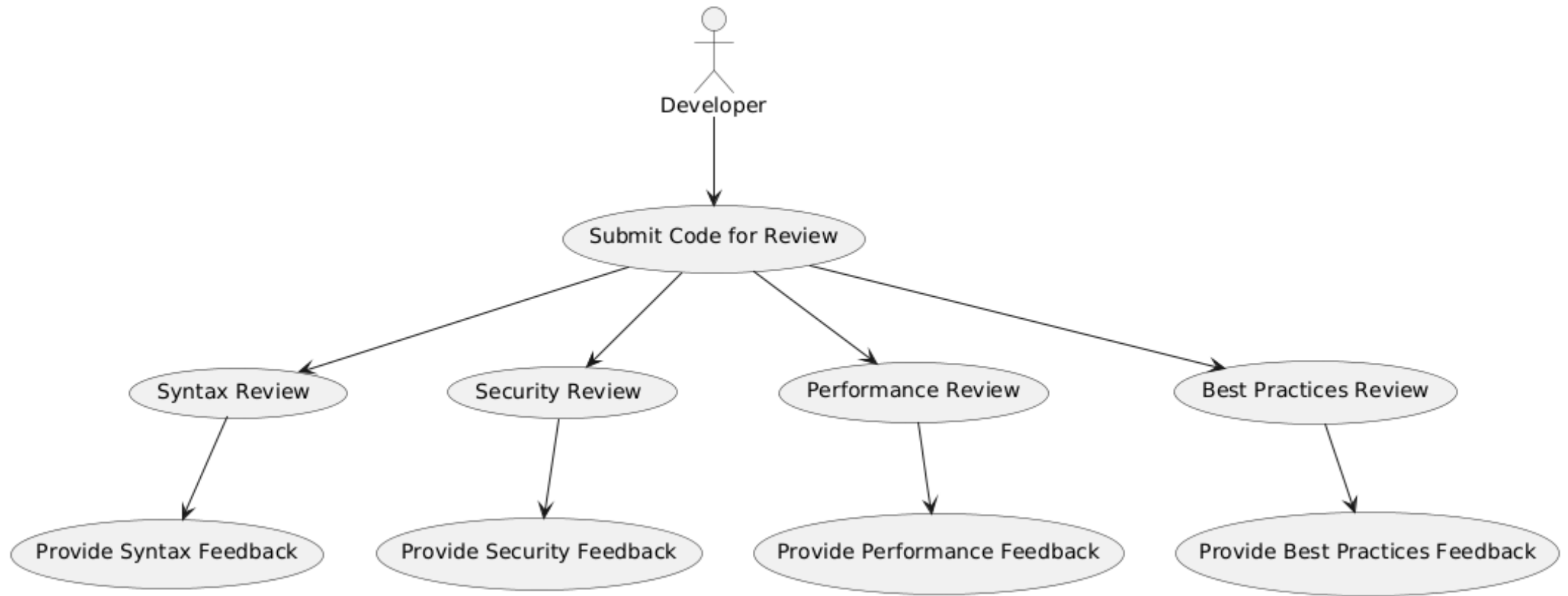


Fig-7: Use case diagram of code review system using GenAI

SEQUENCE DIAGRAM

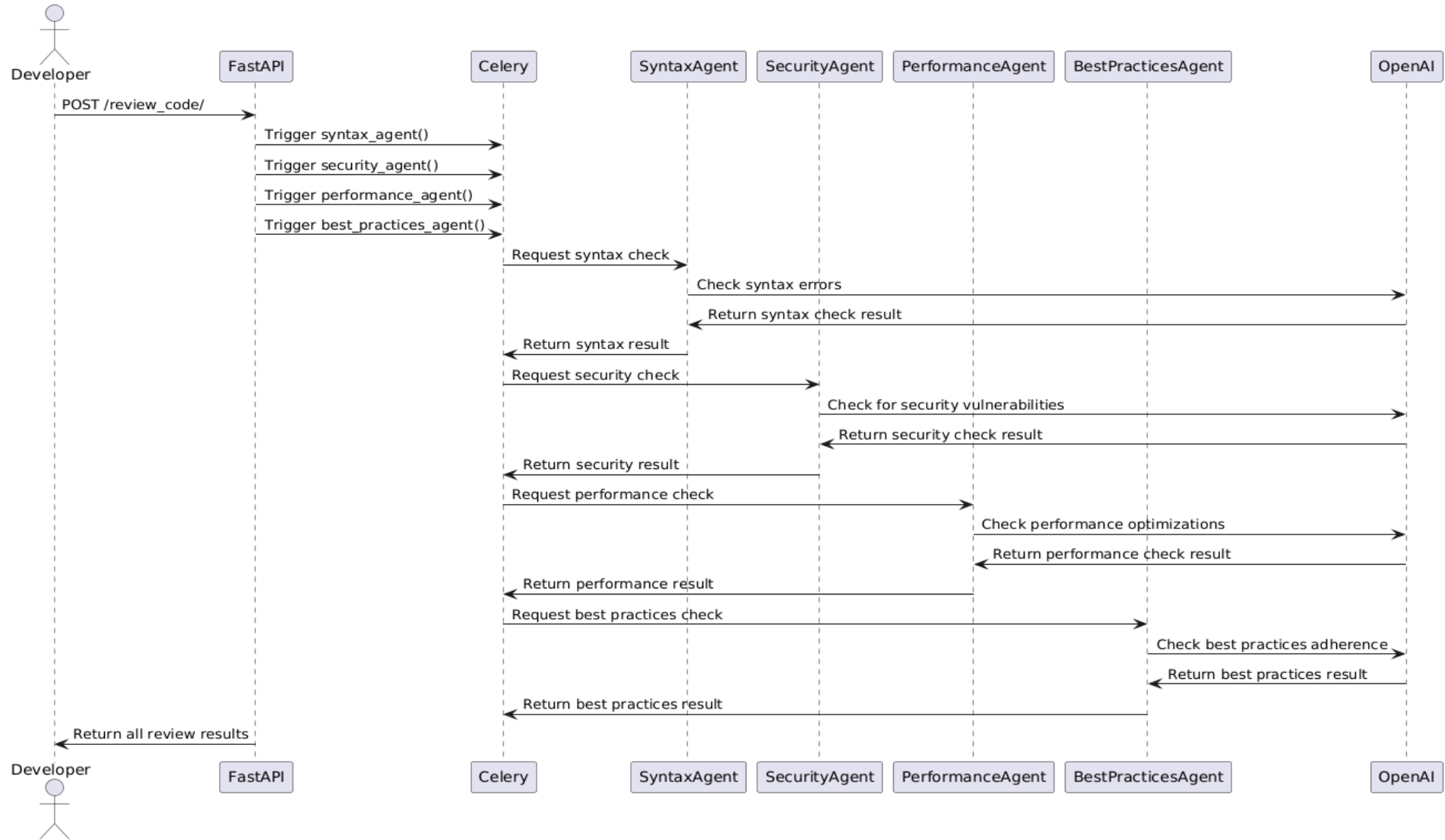
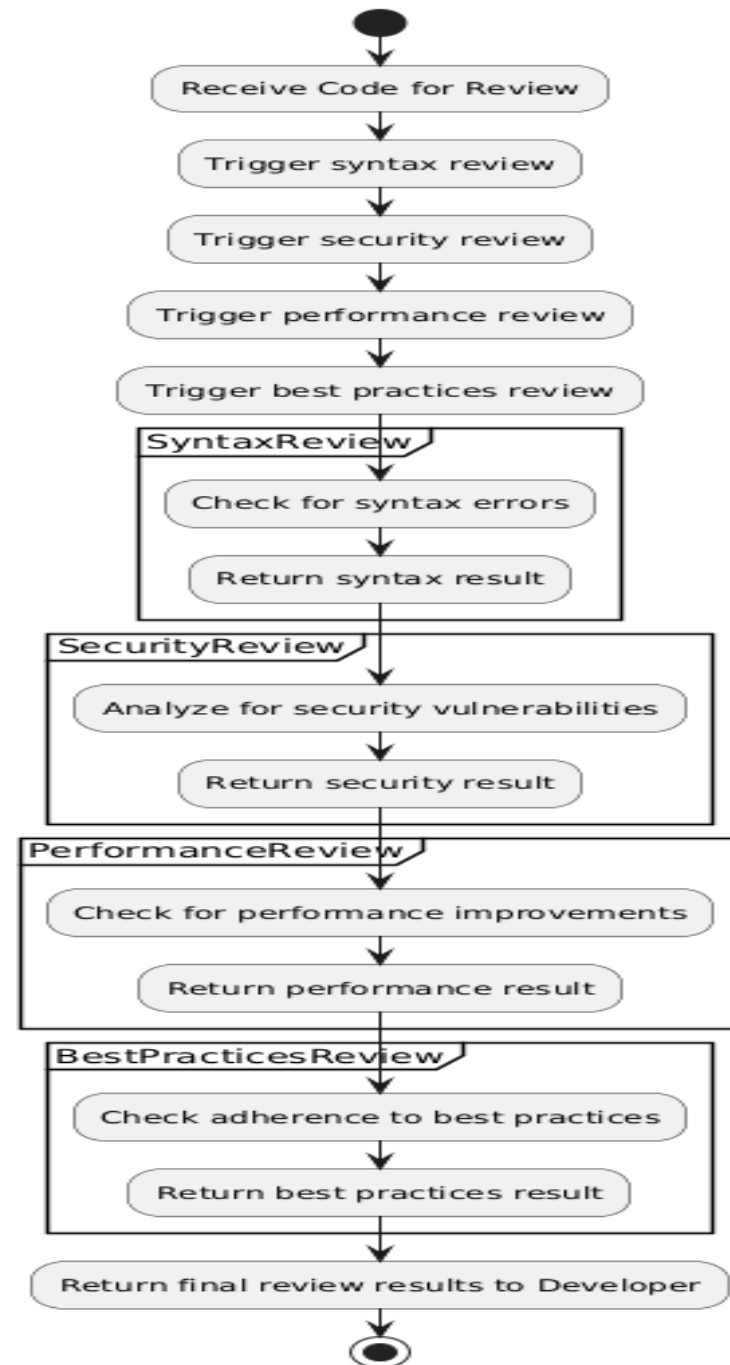


Fig-8: Sequence diagram of code review system using GenAI

ACTIVITY DIAGRAM

Fig-9: Activity diagram of code review system using GenAI



Conclusion

The AI Code Review System revolutionizes the code review process by combining the power of Generative AI with task orchestration and real-time feedback. By leveraging specialized AI agents for syntax, security, performance, and best practices, it provides developers with actionable, consolidated, and context-aware insights. The system's scalable and integrated architecture ensures adaptability to evolving standards, making it an efficient and intelligent tool for improving code quality. This approach addresses the limitations of traditional systems, offering a seamless and future-ready solution for modern software development.

THANK YOU